

Kamar-Taj Lab 01

Library of the Mystic Arts Management System



WONG: You are the new assistant librarian of Kamar-Taj. Here are the keys, a candle, and the rules. Every book must be accounted for. You do not read them.

YOU: Understood. What happened to the last librarian?

WONG: He kept the entire catalogue in his head.

YOU: That is impressive.

WONG: His head is currently in the Mirror Dimension. I would like a catalogue that does not require a head.

YOU: So... a Java program?

WONG: Before the Ancient One's next inspection. The Time Stone can undo many things. It cannot undo an overdue book.

Create a simple Library of the Mystic Arts Management System in Java.

First, read all the requirements before writing any code, and decide what components/classes your project needs. As a hint, the system has to keep track of two different kinds of things, a place that stores them and enforces the rules

connecting them, and the menu that talks to the user. Sketch the fields of each class on paper before you type anything.

Your design should be able to answer three questions at any moment: which tomes exist, which sorcerers are registered, and for each borrowed tome, who is holding it. If your classes can answer those, the operations below are straightforward.

The system should allow the following operations:

1. Register a sorcerer: Add a new borrower to the registry. Each sorcerer should have a unique Sling Ring ID, a name, and a home sanctum (Kamar-Taj, New York, London, or Hong Kong). Only a registered sorcerer may borrow a tome. Officially, there are four sanctums. Rumor says a professor at TUJ is quietly building a fifth. Investigate after you submit; not before.
2. Catalogue a tome: Add a new book to the library. Each book should have a unique VCN, title, and author. (The Vishanti Codex Number (VCN) is my secret attempt to steal the idea of ISBN: a 13-digit number that uniquely identifies every text sanctioned across the multiverse. Books without one are usually forbidden, and always trouble). A VCN must be exactly 13 digits, and the library must reject a tome whose VCN is already catalogued rather than overwriting the existing entry. The same applies to duplicate Sling Ring IDs.
3. Display all tomes: Print the details of every book in the library, so Wong can confirm that nothing has wandered off on its own.
4. Search for a tome: Given a VCN, search and display the details of the book if it exists in the library.
5. Lend out a tome: Given a VCN and a Sling Ring ID, mark the book as borrowed by that sorcerer. If no sorcerer with that ID is registered, display a message saying so and do not lend the book. If the book is already borrowed, display a message indicating that it is unavailable, along with the name of the sorcerer who currently has it.
6. Return a tome: Given a VCN, mark the book as returned and clear the sorcerer holding it. If the book was never borrowed, display a message indicating that it was not checked out (and ask, politely, where they found it). If successful, write thank you for returning this book! Wong has noted that all pages appear to be present.
7. Display borrowed tomes: Print the details of every book currently out of the library, each with the name of the sorcerer holding it. These are the ones we worry about.

Bonus (optional): Given a Sling Ring ID, list every tome that sorcerer currently holds, so Wong knows exactly whose door to knock on.

I have already written the code for a command line interface. Use it (shared on canvas on the page of this assignment) to take input from users and call the appropriate classes to take action.

You have the flexibility to add any functionality you like, and to design the code however you like. Bonus points for creative book titles; no points for opening any portals.

Submit your .java files only!

Adding my interface file to your NetBeans project

1. Download the .java file from Canvas.
2. Open your project in NetBeans. In the Projects window on the left, expand Source Packages until you can see the package that holds your own .java files.
3. Drag the downloaded file onto that package. If dragging does not work, right-click the package, choose New and then Java Class, give it exactly the same name as my file (without .java), then open my file in any text editor and paste its contents over whatever NetBeans created.
4. Either way, the class name inside the file must match the file name exactly, capital letters included. Menu.java must contain class Menu, not class menu.

About the package line

1. The first line of a .java file is often something like package mysticlibrary; that line must name the package the file actually sits in. If you dropped my file into a package with a different name, change that line to match your package.
2. If NetBeans underlines the line in red, click the red bulb in the left margin, it will offer to correct the package for you.
3. If your classes sit directly under Source Packages with no package at all (the default package), delete the package line instead.
4. Keep all of your classes in the same package. Classes in the same package can see each other with no imports; classes in different packages cannot, and that is the error most people get stuck on.

Finding your .java files so you can submit them

1. In the Projects window, right-click the package that holds your .java files, choose Tools, then Locate in System. File Explorer (Windows) or Finder (Mac) opens at that exact folder. The .java files sitting there are the ones you submit. ("Open in System" in the same menu does much the same thing)
2. Do not submit the build or project folders, and do not submit .class files. A .java file is text you can open and read; a .class file is compiled output and cannot be graded. (The .class files live in the build folder, which is why they never appear in the Projects tab).
3. Before you upload, confirm it is the version you want graded. Wong does not accept .class files, and neither do I.

Reminder from Wong:

Rule three: the Ancient One sees every timeline, including the one where you copied this from the internet. Plagiarism, including the use of online codes and/or use of generative AI is strictly prohibited and could lead to an academic dishonesty report. Please refer to the TUJ Plagiarism and cheating policy here: <https://www.tuj.ac.jp/policies/academic-honesty>. Quote: "The penalty for academic dishonesty can vary from receiving a reprimand and a failing grade for a particular assignment, to a failing grade in the course, to suspension or expulsion from the university".

THIS APPLIES TO THE WHOLE GROUP. If one member plagiarises, the sanction is applied to every member, because you submit the work as one team. So if you have any doubt about where a teammate's code came from, ask them directly, and if their answer does not satisfy you, come to me before the deadline or directly after class. Once the work is submitted, it is submitted by all of you.